

Worst-Case Analysis of Online Bipartite Matching Algorithm RANKING

Giovanni Montanari^{1,3}, Nadav Merlis⁴, Simon Mauras¹, and Vianney Perchet^{1,2,3}

¹FairPlay Joint Team, Inria, France

²Criteo AI Lab, Paris, France

³CREST, ENSAE, Institut Polytechnique de Paris

⁴Technion - Israel Institute of Technology

Abstract

This work revisits the celebrated randomized algorithm RANKING for online bipartite matching, known for its simplicity and optimality. Despite its robust performance in the worst case, a natural and fundamental question remains: which instance structures challenge RANKING the most? We address this by analyzing the family of bipartite graphs, paired with arrival orders of the online nodes, that minimize the expected matching size returned by RANKING. Leveraging the differential equation method and introducing a novel and elementary analysis technique on the offline side, we provide a precise performance characterization of RANKING on the so-called *Complete Upper Triangular* instance. Our analysis not only provides a direct and simplified derivation of the expected performance of RANKING on the *CUT* instance, but also reproves that this instance is asymptotically the worst-case input for RANKING as the number of nodes in the graph grows large, a result originally established by Karp et al. [7]. Our approach offers a more elementary and transparent argument, complementing their original proof. These results contribute new insight into the fine-grained behavior of online algorithms and deepen our understanding of adversarial instance design in randomized settings.

1 Introduction

The online matching problem has been extensively studied over the years. In this work, we focus on the online bipartite matching setting. In this classical problem, we are given a bipartite graph $G = G(U, V, E)$, where U represents the set of so-called *online* nodes, V the set of *offline* nodes, and E the set of *edges* between them. In the standard (offline) version of the problem, the goal is simply to find an optimal matching in the graph G , i.e., a matching of maximum cardinality. A matching $M \subseteq E$ is a subset of edges such that no vertex is incident to more than one edge in M . In practice, a matching defines a collection of disjoint pairs.

In the online setting, the problem becomes more challenging. Here, the offline nodes in U are known in advance, while the online nodes in V and their incident edges are revealed sequentially over time. At each time step, a new online node $u \in U$ arrives, and it must be immediately and irrevocably matched to an available offline node $v \in V$ such that $(u, v) \in E$. The objective of an online bipartite matching algorithm is to define a set of rules that govern the real-time assignment of online nodes to offline ones, in order to maximize the size of the resulting matching M , i.e., the total number of matched edges, once all online nodes have arrived.

This problem arises in a variety of real-world applications. A prominent example is the ride-hailing scenario (e.g., Uber), where a centralized system is continuously aware of the locations of available drivers, while ride requests from users arrive sequentially over time. Upon each request, the system must immediately match the user with an available driver, subject to feasibility constraints, such as geographic proximity, since matching a rider with a distant driver would be inefficient. Beyond ride-hailing, matching and online matching problems, with all their variants, are also relevant in many other domains, including online advertising [9, 6], kidney exchange [11, 3], student–university assignment [1, 8], and many more.

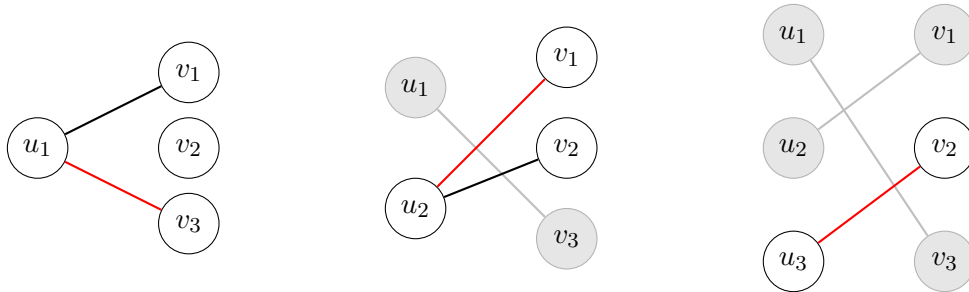


Figure 1: Example of an online matching process.

In Figure 1, we illustrate three stages of an online matching process, where the nodes u_1, u_2, u_3 arrive sequentially and are immediately matched (in red) to an available offline node with which they share an edge. Once matched, each online node disappears from the bipartite graph.

A classical measure of efficiency for online matching algorithms, common across many problems in online computation, is competitive analysis, and in particular, the *competitive ratio*. Intuitively, the competitive ratio of an online matching algorithm is defined as the minimum ratio between the expected size of the matching produced by the online (randomized) algorithm and the size of the matching produced by an offline algorithm, known as the offline optimum (i.e., an algorithm that

has full knowledge of the entire sequence of online nodes and their incident edges in advance). The minimum above is taken over all possible instances, where by instances in the context of bipartite online matching we mean a bipartite graph G and an arrival order of the online nodes U . Formally, the competitive ratio is defined as follows:

Definition 1 (Competitive Ratio for an Online Matching Algorithm). *Let I be an instance (i.e., a bipartite graph along with an arrival order of the online nodes), $ALG(I)$ denote the size of the matching produced by the online algorithm, and $OPT(I)$ the size of the matching produced by the offline optimum. The competitive ratio of the online algorithm is defined as*

$$\inf_I \frac{\mathbb{E}[ALG(I)]}{OPT(I)}.$$

where the expectation is taken over the internal randomness of the algorithm. An algorithm is said to be α -competitive if

$$\frac{\mathbb{E}[ALG(I)]}{OPT(I)} \geq \alpha$$

for every instance I .

Many studies have been conducted in the field of online bipartite matching. Most notably, the milestone work by Karp et al. [7] introduced the RANKING algorithm. In that paper, it is shown that when the arrival order of online nodes is adversarial, RANKING achieves a competitive ratio of $1 - 1/e$. Moreover, it is proven that this performance is optimal: no (randomized) algorithm can achieve a higher competitive ratio in the online bipartite matching problem under adversarial arrivals. The RANKING algorithm has since been studied further and analyzed through simpler or alternative approaches, for example in [2, 4]. In contrast, when online nodes arrive stochastically according to a known distribution, [5] proposes a randomized online algorithm that achieves a competitive ratio strictly greater than $1 - 1/e$ in this stochastic setting.

2 Contributions

To better contextualize the contributions of this work, we first review the foundational results of Karp et al. in [7]. In that seminal paper on online bipartite matching, the RANKING algorithm was introduced for the first time (we will describe its mechanism in detail in the next section). The authors showed that RANKING achieves its worst performance, i.e. its smallest expected matching size, on a specific family of instances known as *upper triangular* instances. Subsequently, they proved that RANKING is $(1 - 1/e)$ -competitive on arbitrary instances. In addition, by analyzing the performance of the simple RANDOM algorithm, which matches each arriving online vertex uniformly at random to an available offline vertex, they showed that, on the so-called *complete upper triangular* instance (a special case within the family of upper triangular instances, whose structure we will formally define in the next section), no online algorithm can achieve a competitive ratio better than $1 - 1/e$, up to terms that vanish as the number of vertices on both sides tends to infinity. This result implies that RANKING is, up to lower-order terms, an optimal algorithm for the online bipartite matching problem.

The main contribution of our work is to provide a simple and direct analysis of RANKING (without relying on the intermediate analysis of RANDOM) on the *complete upper triangular* instance, using the differential equation method by applying Wormald's theorem. A novel aspect of

our approach is that, unlike most analyses in the online matching literature, it is carried out from the perspective of the offline side. Our analysis demonstrates, in a direct and elegant way, that the *complete upper triangular* instance indeed constitutes the worst-case instance for RANKING, at least asymptotically as the number of nodes tends to infinity.

Moreover, we believe that this work can enrich the literature and provide new tools and perspectives for the analysis of randomized online matching algorithms, particularly in the study of their performance on specific instances.

3 Performance of RANKING on *CUT* Instance

3.1 RANKING Algorithm and *CUT* Instance

We begin by explaining the workings of the online bipartite matching algorithm RANKING, introduced in [7]. The mechanism of RANKING is relatively simple and highly intuitive: during the initialization phase, a random ranking is assigned to the offline nodes, which remains fixed throughout the execution of the algorithm. Then, each online node, upon arrival, is matched to the highest-ranked available offline neighbor among those it is connected to. Formally, the pseudo-code of the algorithm appears as:

Algorithm 1 RANKING

- **Input:** A bipartite online graph $G = G(U, V, E)$ with n offline nodes and n online nodes plus the arrival order of the online nodes (U is the online side and V is the offline side).
 - **Initialization phase:** give a random rank to the offline nodes (from 1 to n , 1 is the highest and n is the lowest). The ranks of the offline vertices will be fixed for all the runs of the algorithm.
 - **Matching phase:** For each arriving online vertex $u \in U$:
 Match u with the highest ranked available offline vertex among its edges, so that $(u, v) \in E$ for $v \in V$. If u has no available offline nodes, leave it unmatched.
-

Despite its simplicity, RANKING outperforms every deterministic algorithm as well as the trivial randomized algorithm RANDOM, which matches each arriving online node uniformly at random with an available offline node (the original formulation is in [7]). This superior performance is due to a structural feature of RANKING: it tends to favor offline nodes that have been eligible less frequently in the past. This implicit self-correcting property gives RANKING a degree of robustness, allowing it to perform well even on highly unbalanced instances, something that RANDOM, for instance, fails to achieve.

We now introduce what we previously referred to as the family of *upper triangular* instances. An upper triangular instance consists of a bipartite graph together with an arrival order of the online nodes, such that its adjacency matrix (where rows correspond to offline nodes and columns to online nodes, revealed over time) is upper triangular. That is, entries equal to 1, representing the presence of an edge, appear only above the main diagonal.

The *Complete Upper Triangular (CUT)* instance is a particular case in this family, whose adjacency matrix is a complete upper triangular matrix: it contains 1s on the main diagonal and in all positions above it.

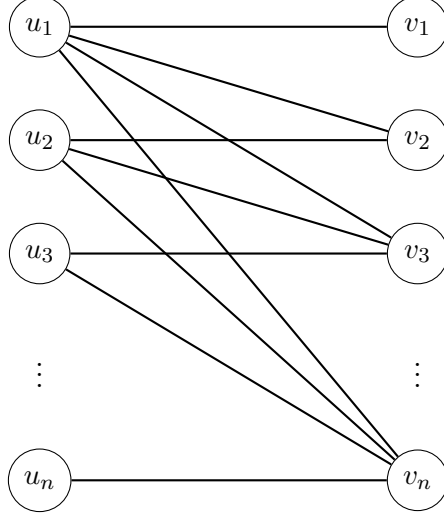


Figure 2: *Complete Upper Triangular (CUT) instance.*

Figure 2 illustrates the structure of the *CUT* instance. The online nodes $u_1, u_2, \dots, u_n$, where n is the total number of nodes on both the online and offline sides, arrive sequentially over time.

It is easy to see that the *CUT* instance admits a *perfect matching*, i.e., a matching in which all nodes are matched. This corresponds to the main diagonal of the adjacency matrix and to the horizontal matching in Figure 2. Therefore, an offline optimal algorithm, one that knows all online nodes and their edges in advance, achieves a matching size of n .

One can also understand why upper triangular instances, and the *CUT* instance in particular, represent worst-case scenarios for RANKING. First, the denominator in the competitive ratio, i.e., the offline optimum, is maximal and equal to n . In addition, each time an online node is matched to an offline node different from its perfect match (for instance, whenever u_i is not matched with v_i in Figure 2), the algorithm effectively loses the opportunity to make a match later in the process. Early-arriving online nodes, which have many possible edges, are highly likely to be matched, but not necessarily to their perfect match. As a result, the late-arriving online nodes, which are sparsely connected, have a low probability of finding an available offline node, since it may have already been matched to a previous online node.

3.2 Analysis

Before moving on to the formal analysis of the competitive ratio of RANKING on the *CUT* instance, it is helpful to outline the reasoning and procedure we will follow. The core idea of our approach is a sequential analysis of the offline side, from v_1 to v_n (see Figure 2), where we examine how the probability that an offline node gets matched evolves as a function of the number of matches that have occurred prior to reaching it in the sequence. For this reason, we define $N(k)$ as the number of offline nodes among $\{v_1, \dots, v_k\}$ that are matched by a run of RANKING. To provide intuition for this sequential analysis, let us consider the following reasoning:

- **v_1** : It has an edge only with the first online vertex u_1 , which will be matched with rank 1 since u_1 is connected to all offline vertices. Therefore, v_1 will be matched only if it has rank

1. Because the ranking on the offline side is assigned uniformly at random, the probability that v_1 is ranked 1 is $\frac{1}{n}$, and thus the probability that v_1 is matched is $\frac{1}{n}$.
- **v_2** : Now, assume the rank of v_1 is known. Let $N(1)$ be the number of matches up to v_1 (i.e., $N(1) = 1$ if v_1 was matched, and $N(1) = 0$ otherwise). There are two cases:
 - If $N(1) = 1$, meaning v_1 had rank 1 and was matched, then v_2 can only be matched if it has rank 2. Since one rank is taken by v_1 , v_2 has $\frac{1}{n-1}$ probability of being matched.
 - If $N(1) = 0$, meaning v_1 was not ranked 1, then v_2 can be matched if it has rank 1 or 2. Out of the $n - 1$ remaining ranks, v_2 has $\frac{2}{n-1}$ probability of being matched.
- **v_3** : Let $N(2)$ be the number of matches up to v_2 (so $N(2)$ can be 0, 1, or 2). There are three cases:
 - If $N(2) = 2$, meaning v_1 and v_2 were ranked 1 and 2 and both matched, then v_3 is matched only if it has rank 3, with probability $\frac{1}{n-2}$.
 - If $N(2) = 1$, meaning exactly one of v_1 or v_2 was matched, then v_3 is matched with probability $\frac{2}{n-2}$. This follows from considering the two cases where either v_1 was matched and v_2 was not, or vice versa.
 - If $N(2) = 0$, meaning neither v_1 nor v_2 was matched, then v_3 is matched if it has rank 1, 2, or 3, so the probability is $\frac{3}{n-2}$.
- The same reasoning applies to all other offline vertices.

This sequential analysis leads to a formal expression for the probability that the offline node v_k , i.e., the k^{th} in the sequential process, is matched, as stated in the following lemma.

Lemma 1. *In the CUT instance, consider the RANKING algorithm analyzed sequentially from the offline side. Let v_k denote the k^{th} offline vertex in the sequential analysis (from v_1 to v_n), let n be the total number of vertices on both sides, $N(k-1)$ the number of matches collected up to step $k-1$ and $\mathcal{F}_{k-1}$ the history of the process up to step $k-1$. Then, the probability that v_k gets matched is given by*

$$\mathbb{P}(v_k \text{ gets matched} \mid \mathcal{F}_{k-1}) = \left\lceil \frac{k - N(k-1)}{n - (k-1)} \right\rceil \wedge 1. \quad (1)$$

Proof. To proceed with the proof, we analyze the ranks and available edges of a generic vertex v_k at two different stages of the algorithm: before the matching phase and during the matching phase.

- **Before the matching phase:**
 - v_k has n available ranks that it can assume (no ranks are known before the analysis begins).
 - v_k has k edges with $\{u_1, u_2, \dots, u_k\}$ (so k potential vertices with which it can have a match).
- **At the $(k-1)$ -step of the matching phase:**
 - v_k has $n - (k-1)$ available ranks (the ranks of $\{v_1, v_2, \dots, v_{k-1}\}$ are already known).

- v_k has still $k - N(k - 1)$ edges with the remaining $k - N(k - 1)$ available online vertices. This is true because, denoting $U_i := \{u \in U : (u, v_i) \in E\}$, the set of neighbors of offline node v_i , then $U_1 \subset U_2 \subset \dots \subset U_k \subset \dots \subset U_n$ in the *CUT* instance.

It is possible now to conclude that among the $n - (k - 1)$ available ranks, v_k will match for the highest (from 1 to n) available $k - N(k - 1)$ ranks. Taking the ratio, we have the formula of the probability, knowing the history of the matching in the sequential analysis up to the offline node v_{k-1} and then $N(k - 1)$. $\square$

Since the formula in Lemma 1 represents a probability, it remains valid only as long as its value is less than or equal to 1, for varying values of k and $N(k)$. A direct implication of Lemma 1 is the following Lemma.

Lemma 2. *$P(v_k \text{ gets matched} \mid \mathcal{F}_{k-1})$ is a non-decreasing sequence of probabilities.*

Proof. To prove the lemma, we compare the following two probabilities:

$$\begin{aligned}\mathbb{P}(v_k \text{ gets matched} \mid \mathcal{F}_{k-1}) &= \left[\frac{k - N(k - 1)}{n - (k - 1)} \right] \wedge 1, \\ \mathbb{P}(v_{k+1} \text{ gets matched} \mid \mathcal{F}_k) &= \left[\frac{k + 1 - N(k)}{n - k} \right] \wedge 1.\end{aligned}$$

We aim to show that:

$$\mathbb{P}(v_{k+1} \text{ gets matched} \mid \mathcal{F}_k) - \mathbb{P}(v_k \text{ gets matched} \mid \mathcal{F}_{k-1}) \geq 0.$$

Let us now focus on the difference between the two fractions, assuming both are less than 1. If we can show that this difference is non-negative, then, since taking the minimum with 1 preserves the order, the lemma will be fully proved.

Let us consider the difference:

$$\frac{k + 1 - N(k)}{n - k} - \frac{k - N(k - 1)}{n - (k - 1)}. \quad (2)$$

By bringing both terms to a common denominator, we obtain:

$$\frac{(n - k + 1)(k + 1 - N(k)) - (n - k)(k - N(k - 1))}{(n - k)(n - k + 1)}.$$

Since $0 \leq k \leq n - 1$, the denominator is positive, so we only need to analyze the numerator. Denote it by N :

$$N = (n - k)(N(k - 1) - N(k)) + (n - k) + (k + 1 - N(k)).$$

Now, by construction of the process, we know that $N(k)$ increases by at most 1 at each step, i.e., either $N(k) = N(k - 1)$ or $N(k) = N(k - 1) + 1$.

Case 1: $N(k) = N(k - 1)$

$$N = (n - k) + (k + 1 - N(k)).$$

Both terms are clearly non-negative: $(n - k) \geq 1$ and $N(k) \leq k$, so $k + 1 - N(k) \geq 1$. Hence, $N \geq 0$.

Case 2: $N(k) = N(k - 1) + 1$

$$N = -(n - k) + (n - k) + (k + 1 - N(k)) = k + 1 - N(k).$$

Again, since $N(k) \leq k$, this expression is non-negative.

Therefore, in both cases, the numerator is non-negative, and hence the difference in (2) is also non-negative. This concludes the proof. $\square$

A direct implication of Lemma 2 is the following: if for some index k^* we have that $\mathbb{P}(v_{k^*} \text{ gets matched} \mid \mathcal{F}_{k^*-1}) = 1$, then for all subsequent offline nodes v_k with $k \geq k^*$ it must hold that $\mathbb{P}(v_k \text{ gets matched} \mid \mathcal{F}_{k-1}) = 1$. This behavior is intuitive: the later vertices in the sequence are those with the highest number of incident edges, and thus the greatest chance of being matched. As a consequence, from node v_{k^*} onward, the algorithm matches every offline vertex with probability 1.

Now we have all the ingredients to write a dynamic evolution equation for the expected value of $N(k)$, the number of matches collected at step k of the sequential offline analysis. To clarify the direction of our reasoning, our ultimate goal is to find the ratio between the expected value $\mathbb{E}[N(n)]$ (the expectation is taken over the internal randomness of RANKING) and n , where n represents the value of the offline optimum assuming a perfect matching exists, as $n \rightarrow \infty$. This ratio precisely corresponds to the average number of matches that the RANKING algorithm collects at the end of the sequential analysis, normalized by the offline optimum, i.e. the competitive ratio.

Hence we can state the theorem that represents the main contribution of this work.

Theorem 1. *The competitive ratio of RANKING in the CUT instance when the number of node n in both online and offline side goes to infinity is $1 - \frac{1}{e}$.*

Proof. At step $k+1$, corresponding to node v_{k+1} , the number of matches collected by the sequential process can either increase by one or remain the same compared to step k (node v_k). Thus, the evolution of the random variable $N(k)$, representing the number of matches at step k , can be described by the recurrence:

$$N(k+1) = N(k) + \begin{cases} 1 & \text{with probability } p = \left\lfloor \frac{k+1-N(k)}{n-k} \right\rfloor \wedge 1 \\ 0 & \text{with probability } 1-p \end{cases}$$

where we used probability $\mathbb{P}(v_{k+1} \text{ gets matched} \mid \mathcal{F}_k)$ in Lemma 1.

Taking expectations conditional on the filtration $\mathcal{F}_k$, representing the history of the process up to step k , we obtain:

$$\mathbb{E}[N(k+1) \mid \mathcal{F}_k] = N(k) + \left[\frac{k+1-N(k)}{n-k} \right] \wedge 1$$

To analyze this process asymptotically, we introduce the normalized variables:

$$t = \frac{k}{n}, \quad n(t) = \frac{N(k)}{n}$$

This rescaling allows us to apply the *differential equation method* via Wormald's Theorem. We now verify that the conditions of Theorem 2 are satisfied:

1. $N(0)/n = 0$,
2. $|N(k+1) - N(k)| \leq 1$.

For Condition 3, we examine the expected increment:

$$\begin{aligned}\mathbb{E}[N(k+1) - N(k) \mid \mathcal{F}_k] &= \left[\frac{n \cdot \left(\frac{k}{n} + \frac{1}{n} - \frac{N(k)}{n} \right)}{n \cdot \left(1 - \frac{k}{n} \right)} \right] \wedge 1 \\ &= \left[\frac{t - n(t)}{1 - t} + \frac{1}{n(1 - t)} \right] \wedge 1\end{aligned}\tag{3}$$

To satisfy Condition 3, we aim to show:

$$|\mathbb{E}[N(k+1) - N(k) \mid \mathcal{F}_k] - f(t, n(t))| \leq cn^{-\lambda}$$

for some constants $c, \lambda > 0$, where $f(t, n) = \left[\frac{t-n}{1-t} \right] \wedge 1$.

The only problematic term is $\frac{1}{n(1-t)}$, which diverges as $t \rightarrow 1$. To control this, we restrict the validity of Wormald's Theorem up to a threshold $t^* < 1$, to be determined later.

Under this assumption, Equation (3) gives:

$$\left| \mathbb{E}[N(k+1) - N(k) \mid \mathcal{F}_k] - \left[\frac{t - n(t)}{1 - t} \right] \wedge 1 \right| \leq \frac{1}{n(1 - t^*)}$$

Thus, Condition 3 holds for all $t \leq t^*$, with $c = \frac{1}{1-t^*}$ and $\lambda = 1$.

We can now apply Wormald's Theorem. The associated differential equation is:

$$\dot{n}(t) = \left[\frac{t - n(t)}{1 - t} \right] \wedge 1, \quad \text{with } n(0) = 0\tag{4}$$

There exists a critical point $t^* < 1$ such that $\dot{n}(t) = 1$ for $t \geq t^*$. Hence, we define:

$$\dot{n}(t) = \begin{cases} \frac{t-n(t)}{1-t}, & \text{if } t \leq t^* \\ 1, & \text{otherwise} \end{cases}$$

Solving the equation for $t \leq t^*$ with $n(0) = 0$, we get:

$$\dot{n}(t) + \frac{n(t)}{1-t} = \frac{t}{1-t}, \quad n(0) = 0\tag{5}$$

with solution:

$$n(t) = t + (1-t) \ln(1-t)\tag{6}$$

To find t^* , we impose $\dot{n}(t^*) = 1$. From Equation (4):

$$\frac{t^* - n(t^*)}{1 - t^*} = 1 \Rightarrow n(t^*) = 2t^* - 1$$

Plugging into Equation (6):

$$2t^* - 1 = t^* + (1 - t^*) \ln(1 - t^*)$$

Solving yields:

$$t^* = 1 - \frac{1}{e}, \quad n(t^*) = 1 - \frac{2}{e}$$

Choosing this value of t^* for condition 3 of Theorem 2, Equation 3 becomes, for all $t \leq t^*$ (and $k \leq k^* = nt^*$):

$$\left| \mathbb{E}[N(k+1) - N(k) \mid \mathcal{F}_k] - \left[\frac{t - n(t)}{1 - t} \right] \wedge 1 \right| \leq \frac{e}{n}$$

Therefore, for $t \in [0, t^*]$, Wormald's theorem guarantees that the normalized process $N(k)/n$ converges in probability (uniformly in k) to the solution $n(t)$ of Equation (4).

More precisely, for every $\beta \in (0, \frac{1}{2})$, $\varepsilon \in (1, 1 - \beta)$, and α such that $(1 + \varepsilon)\beta < \alpha < \varepsilon\beta$, setting $\omega = n^{(1+\varepsilon)\beta}$, and for all $0 \leq k \leq \frac{n}{\omega}$, we have:

$$\mathbb{P} \left(\left| \frac{N(k\omega)}{n} - n \left(\frac{k\omega}{n} \right) \right| \leq k \left(n^{\alpha+\beta} + en^{(1+\varepsilon)\beta-1} + n^{2(1+\varepsilon)\beta-2} \right) \right) \leq k \exp \left(-\frac{n^{2\alpha-(1+\varepsilon)\beta}}{n^2} \right) \quad (7)$$

Equation (7) further implies that, for $t^* = 1 - 1/e$ (we remind that $n(t^*) = 1 - 2/e$), i.e., for $k^* = (1 - 1/e)n$, with high probability the value $N(k^*)/n$ lies between the following lower and upper bounds from $n(t^*)$:

$$\frac{N(k^*)}{n} \geq n(t^*) - \Delta_n^* \quad \text{with high probability} \quad (8)$$

$$\frac{N(k^*)}{n} \leq n(t^*) + \Delta_n^* \quad \text{with high probability} \quad (9)$$

where, for notational simplicity, we define Δ_n^* as:

$$\Delta_n^* = \frac{k^*}{\omega} \left(n^{\alpha+\beta} + en^{(1+\varepsilon)\beta-1} + n^{2(1+\varepsilon)\beta-2} \right)$$

Moreover, we know that at $t^* = 1 - \frac{1}{e}$, and consequently at $k^* = (1 - \frac{1}{e})n$, the derivative $\dot{n}(t^*)$ equals 1. Since $\dot{n}(t^*)$ represents the conditional probability $\mathbb{P}(v_{k^*} \text{ gets matched} \mid \mathcal{F}_{k^*-1})$, it follows that, with the same high probability given by Equation 7:

$$1 - \Delta_n^* \leq \mathbb{P}(v_{k^*} \text{ gets matched} \mid \mathcal{F}_{k^*-1}) \leq 1$$

By Lemma 2, we know that the sequence of probabilities $\mathbb{P}(v_k \text{ gets matched} \mid \mathcal{F}_{k-1})$ is non-decreasing in k . Therefore, if $\mathbb{P}(v_{k^*} \text{ gets matched} \mid \mathcal{F}_{k^*-1}) \geq 1 - \Delta_n^*$, then all subsequent probabilities for $k \geq k^*$ must also be greater than or equal to $1 - \Delta_n^*$.

As a consequence, all the remaining $n - (1 - 1/e)n$ offline nodes (i.e., those arriving after v_{k^*}) will be matched with high probability, and will contribute to the competitive ratio with something greater than or equal to:

$$\frac{[n - (1 - 1/e)n](1 - \Delta_n^*)}{n} = \frac{1}{e}(1 - \Delta_n^*)$$

and less than or equal to:

$$\frac{[n - (1 - 1/e)n]}{n} = \frac{1}{e}$$

Using this together with Equations (8) and (9), namely by separating the two contributions to the competitive ratio (the first from 1 to k^* and the second from k^* to n), we can extend the bounds on $N(k^*)/n$ to bounds on $N(n)/n$. In particular, we obtain that, with high probability:

$$\begin{aligned} \frac{N(n)}{n} &\geq n(t^*) - \Delta_n^* + \frac{1}{e}(1 - \Delta_n^*) \\ &= 1 - \frac{2}{e} + \frac{1}{e}(1 - \Delta_n^*) - \Delta_n^* \\ &= 1 - \frac{1}{e} - (1 + \frac{1}{e})\Delta_n^* \end{aligned} \tag{10}$$

$$\begin{aligned} \frac{N(n)}{n} &\leq n(t^*) + \Delta_n^* + \frac{1}{e} \\ &= 1 - \frac{1}{e} + \Delta_n^* \end{aligned} \tag{11}$$

Equations (10) and (11) define, with high probability, a confidence interval for $\frac{N(n)}{n}$ around $1 - \frac{1}{e}$. Since the error term Δ_n^* vanishes as $n \rightarrow \infty$, it follows that

$$\frac{N(n)}{n} \xrightarrow{P} 1 - \frac{1}{e}.$$

In essence, we first apply Wormald's theorem to establish that the discrete stochastic process converges in probability to the solution of the corresponding ODE up to a fixed k^* . Then, exploiting the monotonicity of the transition probability given by Lemma 2, we extend these bounds from k^* to n , thereby proving that $N(n)/n$ converges in probability to $1 - 1/e$.

Given that $N(n)/n$ converges in probability and is bounded between 0 and 1 for all n , convergence in probability implies convergence in L^1 . Therefore, we obtain

$$\lim_{n \rightarrow \infty} \mathbb{E} \left[\frac{N(n)}{n} \right] = 1 - \frac{1}{e}.$$

Equivalently,

$$\lim_{n \rightarrow \infty} \frac{\mathbb{E}[N(n)]}{n} = 1 - \frac{1}{e},$$

so the asymptotic competitive ratio of RANKING on the *CUT* instance equals $1 - \frac{1}{e}$.

This completes the proof. $\square$

Essentially, we applied the *differential equation method*, leveraging the guarantees provided by Wormald’s theorem, to transition from analyzing the discrete stochastic process $N(k)$ to studying a simple ODE whose solution $n(t)$ asymptotically matches, as $n \rightarrow \infty$, the behavior of the original process. We carried out this analysis up to $t^* = 1 - 1/e$. Beyond this point, we invoked the monotonicity Lemma 2 to conclude that, starting from the step k^* corresponding to t^* , every offline node would be matched. This yields, in the end, that $\mathbb{E}[N(n)]/n$, the competitive ratio of RANKING on the *Complete Upper Triangular* instance, converges, as $n \rightarrow \infty$, to $1 - 1/e$.

Combining this result with the known fact from Karp et al. [7] that RANKING is $(1 - 1/e)$ -competitive, meaning there is no instance where RANKING achieves a competitive ratio lower than $1 - 1/e$, we conclude that, at least for sufficiently large n , the *CUT* instance represents the worst-case instance for RANKING, thus confirming both the conjecture by Karp et al. and a natural intuition.

4 Conclusions and Future Directions

This paper focused on the celebrated randomized online bipartite matching algorithm RANKING. Starting from a conjecture by Karp et al. [7], which hypothesized that the worst-case instance for RANKING is the so-called *Complete Upper Triangular* instance, we studied the performance of RANKING on this instance. Using the differential equation method together with a simple yet effective sequential analysis on the offline side, which led to an exact formulation of the probability that a node at a certain position gets matched, we characterized the asymptotic performance of RANKING on the *CUT* instance. This analysis showed that when n , the number of nodes on both sides of the graph, is sufficiently large, the competitive ratio of RANKING on the *CUT* instance is exactly $1 - 1/e$, thereby proving that, at least asymptotically, the *CUT* instance represents the worst-case scenario for RANKING.

Beyond presenting a simple and elegant analysis of the worst-case instances for RANKING, thus enriching the literature through the methods used and alternative perspectives for analyzing randomized algorithms on fixed instances, several interesting future directions can still be outlined.

First, one could attempt to prove that the *CUT* instance is the worst-case for all values of the number of nodes n , and not only asymptotically. Such a result would fully prove the conjecture by Karp et al. [7]. In this context, it would also be very interesting to understand how the competitive ratio of RANKING varies as a function of n , aiming to find a lower bound for finite values rather than just asymptotic ones. Finally, it would be worthwhile to prove, supported already by intuitive evidence, that the *CUT* instance represents the worst possible instance not only for RANKING but for all randomized algorithms.

References

- [1] Eduardo M. Azevedo and Jacob D. Leshno. A supply and demand framework for two-sided matching markets. *Journal of Political Economy*, 124(5):1235–1268, 2016.

- [2] Benjamin E. Birnbaum and Claire Mathieu. On-line bipartite matching made simple. *SIGACT News*, 39(1):80–87, 2008.
- [3] Péter Biró, David F. Manlove, and Romeo Rizzi. Maximum weight cycle packing in directed graphs, with application to kidney exchange programs. *Discrete Mathematics, Algorithms and Applications*, 1(04):499–517, 2009.
- [4] Nikhil R. Devanur, Kamal Jain, and Robert D. Kleinberg. Randomized primal-dual analysis of RANKING for online bipartite matching. In *Proceedings of the 14th ACM Conference on Electronic Commerce (EC)*, pages 137–144. SIAM, 2013.
- [5] Jon Feldman, Aranyak Mehta, Vahab S. Mirrokni, and S. Muthukrishnan. Online stochastic matching: Beating $1-1/e$. *CoRR*, abs/0905.4100, 2009.
- [6] Gagan Goel and Aranyak Mehta. Online budgeted matching in random input models with applications to adwords. In *Proceedings of the Nineteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA ’08, page 982–991, USA, 2008. Society for Industrial and Applied Mathematics.
- [7] Richard M. Karp, Umesh V. Vazirani, and Vijay V. Vazirani. An optimal algorithm for on-line bipartite matching. In *Proceedings of the 22nd Annual ACM Symposium on Theory of Computing (STOC)*, pages 352–358. ACM, 1990.
- [8] Jacob D Leshno. Dynamic matching in school choice: Efficient seat reallocation after late cancellations. *Operations Research*, 69(6):1834–1854, 2021.
- [9] Aranyak Mehta, Amin Saberi, Umesh Vazirani, and Vijay Vazirani. Adwords and generalized online matching. *J. ACM*, 54(5):22–es, October 2007.
- [10] Nathan Noiry, Vianney Perchet, and Flore Sentenac. Online matching in sparse random graphs: Non-asymptotic performances of greedy algorithm. In A. Beygelzimer, Y. Dauphin, P. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, 2021.
- [11] Alvin Roth, Tayfun Sönmez, and Utku Ünver. Kidney exchange. *The Quarterly Journal of Economics*, 119(2):457–488, 2004.
- [12] Nicholas C. Wormald. Differential equations for random processes and random graphs. *Annals of Applied Probability*, 5(4):1217–1235, 1995.
- [13] Nicholas C. Wormald. The differential equation method for random graph processes and greedy algorithms. In Miklós Karoński and Andrzej Ruciński, editors, *Lectures on Approximation and Randomized Algorithms*, pages 73–155. PWN, 1999.
- [14] Nicholas C. Wormald. Analysis of greedy algorithms on graphs with bounded degrees. *Discrete Mathematics*, 273:235–260, 2003.

A Simulations

We have done simulations on the functioning of RANKING on the *CUT* instance, collecting the number of matches obtained for different values of n . For each distinct value of n , the number of matches obtained was calculated as an average over 80 different algorithm runs. This was necessary

to represent the algorithm's performance more efficiently, avoiding fluctuations from the random ranking given at the beginning of each run. The average number of matches obtained was then divided by the value of n (the one that achieves the offline optimum) in order to obtain a plot of the competitive ratio as a function of n .

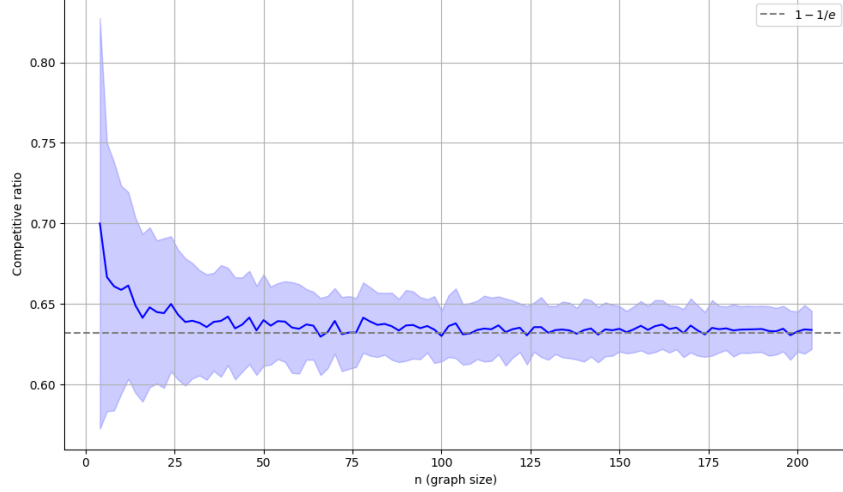


Figure 3: Competitive Ratio as a function of n .

Figure 3 shows the average number of matches obtained by RANKING on the *CUT* instance, normalized by n , along with the corresponding standard deviation. The dashed line marks the value $1 - 1/e$. It's possible to notice how, for low values of n , the algorithm collects a larger number of matches. This is because the randomness of RANKING has less influence. However, as soon as n starts to increase, the competitive ratio rapidly tends towards its theoretical limit value of $1 - \frac{1}{e}$, as we would expect.

The simulation code is available at the anonymous link <https://anonymous.4open.science/r/RANKING-on-CUT-Simulations-7A58/>.

B Differential Equation Method and Wormald's Theorem

Theorem 2. For all $n \geq 1$, let $Y(k) = Y^{(n)}(k)$ be a Markov chain with respect to a filtration $\{\mathcal{F}_k\}_{k \geq 1}$. Suppose that there exists a function f such that:

- $Y(0)/n = z(0)$;
- $|Y(k+1) - Y(k)| \leq n^\beta$;
- $|\mathbb{E}[Y(k+1) - Y(k) \mid \mathcal{F}_k] - f(k/n, Y(k))| \leq cn^{-\lambda}$,

for some constant $c > 0$, where $0 < \beta < \frac{1}{2}$ and $\lambda > 0$. Suppose that the following differential equation with initial condition $z(0)$ has a unique solution z :

$$\frac{d}{dt}z(t) = f(t, z(t)).$$

Then, the rescaled process $Y(\lfloor tn \rfloor)/n$ converges in probability to $z(t)$ for the topology of uniform convergence.

More precisely, for every $1 < \varepsilon < 1 - \beta$, for every $(1 + \varepsilon)\beta < \alpha < \varepsilon\beta$, and for every $0 \leq k \leq \frac{n}{n^{(1+\varepsilon)\beta}}$, letting $\omega = n^{(1+\varepsilon)\beta}$, it holds that:

$$\mathbb{P} \left(\left| \frac{Y(k\omega)}{n} - z(i\omega/n) \right| \leq k \left(n^{\alpha+\beta} + cn^{(1+\varepsilon)\beta-\lambda} + n^{2(1+\varepsilon)\beta-2} \right) \right) \leq k \exp \left(-\frac{n^{2\alpha-(1+\varepsilon)\beta}}{n^2} \right).$$